**DBMS – ADVANCED SQL QUERY SYSTEM**

**Project Title: Musical Instruments E-Commerce Management System**

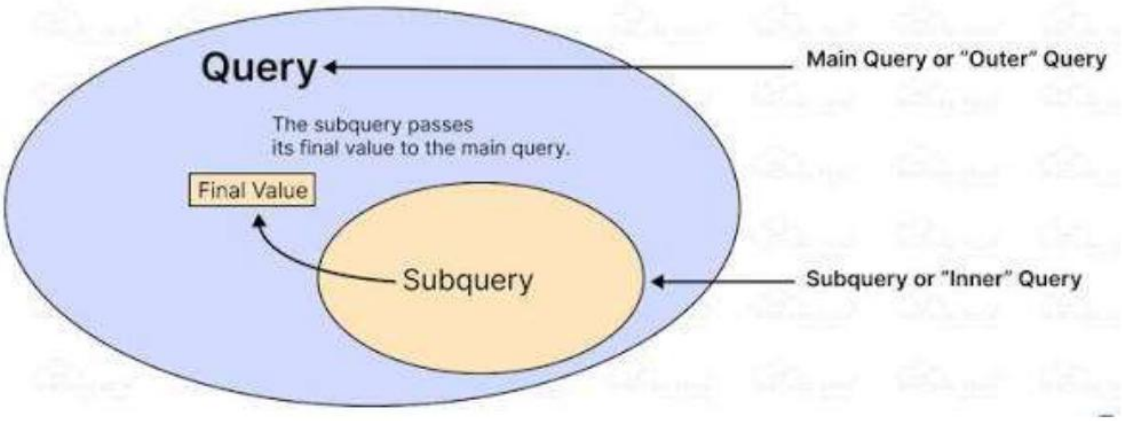
**Objectives**

1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

**1. Subqueries and Nested Queries**

A **subquery** is a query written inside another SQL query.

The inner query is executed first, and its result is used by the outer query.



**Basic Structure**

```sql
SELECT
    order_id,
    order_date,
    customer_id
FROM
    sales.orders
WHERE
    customer_id IN (
        SELECT
            customer_id
        FROM
            sales.customers
        WHERE
            city = 'New York'
    )
ORDER BY
    order_date DESC;
```

outer query

subquery

**Types of Subqueries**

*Type* Description

| |
|---|
| *Single-row subquery* - Returns one value |
| *Multiple-row subquery* - Returns multiple values |
| *Nested subquery* - A subquery inside another subquery |
| *Correlated subquery* - Inner query depends on the outer query |

Example

Find orders whose amount is greater than the average order amount.

```
SQL> SELECT Order_ID, Customer_ID, Total_Amount
  2  FROM Orders
  3  WHERE Total_Amount >
  4        (SELECT AVG(Total_Amount)
  5         FROM Orders);

  ORDER_ID CUSTOMER_ID TOTAL_AMOUNT
---------- ----------- ------------
      1002         203        45000
      1004         202        38000
      1005         205        28000
      1006         206        42000
```

Explanation

The inner query:

SELECT AVG(Total_Amount) FROM Orders;

calculates the average order amount.

The outer query then displays orders whose Total_Amount is greater than that average.

**2. Find Products Above Average Price**

A subquery can be used to compare each product's price with the average product price.

Assuming the Product table contains:

• Product_ID

• Product_Name

• Price

Query

```
SQL> SELECT
  2      Product_ID,
  3      Product_Name,
  4      Price
  5  FROM Product
  6  WHERE Price >
  7        (SELECT AVG(Price)
  8         FROM Product);

PRODUCT_ID PRODUCT_NAME                PRICE
---------- ------------------------ ----------
       102 Stratocaster Electric         45000
       103 Digital Piano P45             38000
       105 Casio CDP-S100                32000
       106 Roland TD-1K Drum Kit         42000
       107 Pearl Roadshow Drums          28000
       110 Fender Squier Bass            22000

6 rows selected.
```

Explanation

The inner query calculates the average price:

SELECT AVG(Price) FROM Product;

The outer query displays products whose price is greater than the calculated average.

*The actual output depends on the Product data in the database.*

### 2.1 Products Below Average Price

The same concept can be used to find products below the average price.

```
SQL> SELECT
  2      Product_ID,
  3      Product_Name,
  4      Price
  5  FROM Product
  6  WHERE Price <
  7        (SELECT AVG(Price)
  8         FROM Product);

PRODUCT_ID PRODUCT_NAME                PRICE
---------- ------------------------ ----------
       101 Acoustic Guitar F310          12500
       104 Casio CT-S300                  9500
       108 Yamaha PSR-E373               14500
       109 Classical Guitar C40           7500
       111 Korg Mini Keyboard             6500
       112 Cajon Box Drum                 5500

6 rows selected.
```

### 3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase amount.

Step 1: Calculate Customer-wise Purchase.

```
SQL> SELECT
  2      Customer_ID,
  3      SUM(Total_Amount) AS Total_Purchase
  4  FROM Orders
  5  GROUP BY Customer_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        202          50500
        203          50500
        204           9500
        205          28000
        206          42000
        207          14500
        208          22000
        209           6500

8 rows selected.
```

Step 2: Find the Maximum Purchase.

```
SQL> SELECT MAX(Total_Purchase)
  2  FROM
  3  (
  4      SELECT
  5          Customer_ID,
  6          SUM(Total_Amount) AS Total_Purchase
  7      FROM Orders
  8      GROUP BY Customer_ID
  9  );

MAX(TOTAL_PURCHASE)
-------------------
              50500
```

The inner query calculates the total purchase for each customer.

The outer query finds the maximum value among those totals.

**3.1 Display Customer Details with Maximum Purchase.**

```
SQL> SELECT
  2      c.CustomerID,
  3      c.CustomerName,
  4      SUM(o.Total_Amount) AS Total_Purchase
  5  FROM Customer c
  6  JOIN Orders o
  7      ON c.CustomerID = o.Customer_ID
  8  GROUP BY
  9      c.CustomerID,
 10      c.CustomerName
 11  HAVING SUM(o.Total_Amount) = (
 12      SELECT MAX(Total_Purchase)
 13      FROM (
 14          SELECT SUM(Total_Amount) AS Total_Purchase
 15          FROM Orders
 16          GROUP BY Customer_ID
 17      ));

CUSTOMERID CUSTOMERNAME    TOTAL_PURCHASE
---------- --------------- --------------
       202 Madhuu                  50500
       203 alia                    50500
```

Explanation

**The innermost query:**

SELECT SUM(Total_Amount)

FROM Orders

GROUP BY Customer_ID;

calculates the purchase amount for every customer.

**The next query:**

SELECT MAX(Total_Purchase)

finds the highest purchase amount.

The main query displays the customer whose purchase amount matches that maximum.

**4. Complex Business Queries**

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

**4.1 Customers Who Placed Orders Above Average Order Value**

```
SQL> SELECT
  2      c.CustomerID,
  3      c.CustomerName,
  4      o.Order_ID,
  5      o.Total_Amount
  6  FROM Customer c
  7  JOIN Orders o
  8      ON c.CustomerID = o.Customer_ID
  9  WHERE o.Total_Amount >
 10  (
 11      SELECT AVG(Total_Amount)
 12      FROM Orders
 13  )
 14  ORDER BY o.Total_Amount DESC;

CUSTOMERID CUSTOMERNAME      ORDER_ID TOTAL_AMOUNT
---------- --------------- ---------- ------------
       203 alia                  1002        45000
       206 Sana                  1006        42000
       202 Madhuu                1004        38000
       205 Kiran                 1005        28000
```

**Purpose**

This report identifies customers who placed orders whose value is greater than the overall average order value.

### 5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated from Order_Details.

```
SQL> SELECT
  2      p.Product_ID,
  3      p.Product_Name,
  4      SUM(od.Subtotal) AS Total_Sales
  5  FROM Product p
  6  JOIN Order_Details od
  7      ON p.Product_ID = od.Product_ID
  8  GROUP BY
  9      p.Product_ID,
 10      p.Product_Name
 11  HAVING SUM(od.Subtotal) > (
 12      SELECT AVG(Product_Sales)
 13      FROM (
 14          SELECT SUM(Subtotal) AS Product_Sales
 15          FROM Order_Details
 16          GROUP BY Product_ID
 17      ))
 18  ORDER BY Total_Sales DESC;

PRODUCT_ID PRODUCT_NAME              TOTAL_SALES
---------- ------------------------ -----------
       102 Stratocaster Electric          45000
       106 Roland TD-1K Drum Kit          42000
       103 Digital Piano P45              38000
       107 Pearl Roadshow Drums           28000
```

Purpose

This query identifies products whose total sales are greater than the average sales generated by the products.

### 6. Customer Purchase Report

A detailed customer purchase report can combine multiple tables.

```
SQL> SELECT
  2      c.CustomerID,
  3      c.CustomerName,
  4      COUNT(o.Order_ID) AS Total_Orders,
  5      SUM(o.Total_Amount) AS Total_Purchase,
  6      AVG(o.Total_Amount) AS Average_Order_Value
  7  FROM Customer c
  8  JOIN Orders o
  9      ON c.CustomerID = o.Customer_ID
 10  GROUP BY
 11      c.CustomerID,
 12      c.CustomerName
 13  ORDER BY Total_Purchase DESC;

CUSTOMERID CUSTOMERNAME    TOTAL_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_VALUE
---------- --------------- ------------ -------------- -------------------
       202 Madhuu                     2          50500               25250
       203 alia                       2          50500               25250
       206 Sana                       1          42000               42000
       205 Kiran                      1          28000               28000
       208 Priya                      1          22000               22000
       207 Rahul                      1          14500               14500
       204 Ranbir                     1           9500                9500
       209 Vikram                     1           6500                6500

8 rows selected.
```

Report Contains

• Customer ID

• Customer Name

• Number of orders

• Total purchase amount

• Average order value

**7. Advanced Product Sales Report**

This report combines product and order details.

```
SQL> SELECT c.CustomerID, c.CustomerName,
  2         NVL(SUM(o.Total_Amount),0) AS Total_Spent,
  3         CASE WHEN SUM(o.Total_Amount) >= 40000 THEN 'Gold'
  4              WHEN SUM(o.Total_Amount) >= 15000 THEN 'Silver'
  5              WHEN SUM(o.Total_Amount) > 0      THEN 'Bronze'
  6              ELSE 'Inactive' END AS Segment
  7  FROM Customer c
  8  LEFT JOIN Orders o ON c.CustomerID = o.Customer_ID
  9  GROUP BY c.CustomerID, c.CustomerName
 10  ORDER BY Total_Spent DESC;

CUSTOMERID CUSTOMERNAME    TOTAL_SPENT SEGMENT
---------- --------------- ----------- --------
       202 Madhuu                50500 Gold
       203 alia                  50500 Gold
       206 Sana                  42000 Gold
       205 Kiran                 28000 Silver
       208 Priya                 22000 Silver
       207 Rahul                 14500 Bronze
       204 Ranbir                 9500 Bronze
       209 Vikram                 6500 Bronze

8 rows selected.
```